P0521r0
Proposed Resolution for CA 14 (shared_ptr use_count/unique)

Published Proposal, 11 November 2016

This version:
http://wg21.link/P0521

Author:
Stephan T. Lavavej (Microsoft)

Audience:
SG1, LEWG, LWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

This paper proposes a resolution to C++17 CD comment CA 14, which identified issues with `use_count` and `unique` in `shared_ptr`.

§ 1. CA 14

> The removal of the "debug only" restriction for `use_count` and `unique` in `shared_ptr` introduced a bug: in order for `unique` to produce a useful and reliable value, it needs a synchronize clause to ensure that prior accesses through another reference are visible to the successful caller of `unique`. Many current implementations use a relaxed load, and do not provide this guarantee, since it's not stated in the Standard. For debug/hint usage that was OK. Without it the specification is unclear and misleading.

§ 2. Proposed Wording

The proposed changes are relative to [N4604], the Committee Draft for C++17.

The ❡ character is used to denote a placeholder number which the editor shall determine.

§ 2.1. Part A

Change 20.11.2.2.5 **[util.smartptr.shared.obs]** as depicted:

```
long use_count() const noexcept;
```

-7- *Returns:* The number of `shared_ptr` objects, `*this` included, that share ownership with `*this`, or 0 when `*this` is empty.

-➀- *Synchronization:* None.

-➀- [*Note:* `get() == nullptr` does not imply a specific return value of `use_count()`. — *end note*]

-➀- [*Note:* `weak_ptr<T>::lock()` can affect the return value of `use_count()`. — *end note*]

-➀- [*Note:* When multiple threads can affect the return value of `use_count()`, the result should be treated as approximate. In particular, `use_count() == 1` does not imply that accesses through a previously destroyed `shared_ptr` have in any sense completed. — *end note*]

```
bool unique() const noexcept;
```

-8- *Returns:* `use_count() == 1`.

~~-9- [*Note:* If you are using `unique()` to implement copy on write, do not rely on a specific value when `get() == nullptr`. — *end note*]~~

§ 2.2. Part B

Change 20.11.2.2 **[util.smartptr.shared]**/1 as depicted:

```
namespace std {
  template<class T> class shared_ptr {
  public:
    [...]
    long use_count() const noexcept;
    bool unique() const noexcept;
    explicit operator bool() const noexcept;
    [...]
  };
  [...]
} // namespace std
```

Change 20.11.2.2.5 **[util.smartptr.shared.obs]** as depicted:

~~bool unique() const noexcept;~~

~~-8- *Returns:* use_count() == 1.~~

Add a new section:

D.➀ Deprecated `shared_ptr` observers [depr.util.smartptr.shared.obs]

The following member is defined in addition to those specified in **[util.smartptr.shared]**:

```
namespace std {
  template<class T> class shared_ptr {
  public:
    bool unique() const noexcept;
  };
}
```

```
bool unique() const noexcept;
```

Returns: `use_count() == 1`.

§ References

§ Informative References

[N4604]
Richard Smith. C++17 CD Ballot Document. 12 July 2016. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4604.pdf